End-to-End Superpod Modeling for Photonic Interconnects: A Lightmatter Passage Case Study on Gemini-Class TPU Workloads

Pranav Sitaraman
6.5930 Hardware Architecture for Deep Learning

Abstract

This work evaluates Lightmatter-style photonic interconnects as an emerging technology for DNN accelerators. AccelForge is used as the local accelerator mapper, then extended upward with a topology-aware network model, an iterative logical-memory feedback loop, a distributed mapping selector, a compiler/dataflow strategy model, and a full-SuperPod model for Gemini-class workloads. The main finding is not merely that lower-pJ/bit links reduce energy. The more interesting result is that photonic properties–high bandwidth, flat switched topology, and native multicast–make different compiler and memory-placement strategies attractive. In 16-node AccelForge studies, an EVK100 optical crossbar reduces energy by up to 72.21% and latency by up to 97.57% relative to an electrical mesh. In a 64-chip TPU-cube traffic model, M1000 switched photonics reduces network energy by up to 99.91%. At full 9216-chip Ironwood/TPU7x scale, a Gemini-class long-context proxy reaches 14.50× modeled inference latency speedup with the full M1000 photonic compiler stack, while training improves by 1.43× because it becomes compute dominated once collectives are reduced. The results are tied to public assumptions, explicit equations, and validation checks for the constants, topology behavior, mapping loop, and end-to-end summaries.

I. Project Overview

A. Project category and high-level goal

This project is based on Project 12, "Evaluate a New Technology," and on the AccelForge cluster-mapping example. The selected technology is co-packaged photonic interconnect, modeled using publicly available information about the Lightmatter Passage family: EVK50, EVK100, L200, L200X, and M1000 [1]–[4]. The goal is to answer a system-level question:

> *If a TPU-class accelerator cluster replaces electrical ICI with Lightmatter-style photonic interconnects, does the accelerator simply get cheaper links, or do new interconnect properties change the optimal topology, memory hierarchy, compiler dataflow, and end-to-end workload behavior?*

The methodology therefore goes beyond a constant-factor link substitution. It starts with a network-aware AccelForge study and then extends it to a full-superpod model for Gemini-class training and inference on the public 9216-chip Ironwood/TPU7x pod [8]. Because Google does not disclose Gemini 3/3.1 parameter counts or expert layouts, the superpod study uses explicit public proxy workloads, not confidential model internals [12]–[15]. This distinction is important: the proxy models are stress tests for dense, sparse MoE, and long-context regimes, not claims about Google's proprietary architecture.

B. Motivation

Modern large-language-model systems are no longer limited only by matrix multiply throughput. Distributed inference and training depend on how quickly the system can move activations, KV-cache state, expert tokens, gradients, optimizer state, and shared weights across many accelerators. TPU v5p and Ironwood/TPU7x expose this directly: TPU systems are organized around high-speed ICI, 3-D torus connectivity, and large pod-scale fabrics [7]–[9]. Electrical links work well for local, nearest-neighbor communication, but they become expensive for traffic patterns that pay repeated hop cost or repeated fanout cost.

Photonic interconnects change several properties at once:

- **Energy per bit.** EVK100 is modeled at 1.9 pJ/bit, while the electrical baseline is modeled at 10 pJ/bit. M1000 is modeled at 2.3 pJ/bit including laser overhead [4].
- **Bandwidth.** The modeled one-way bandwidth range grows from 800 Gb/s electrical to 57.3 Tb/s for M1000.
- **Topology.** A photonic switched fabric can approximate single-hop any-to-any communication rather than multi-hop torus routing.
- **Multicast.** Optical wavelength routing and switching make broadcast/cache-fill patterns fundamentally cheaper than repeated electrical unicast.
- **Compiler opportunity.** If remote memory and multicast are cheap, XLA/GSPMD/Pallas-like compilers should prefer different sharding, prefetching, caching, and expert-placement choices [11].

The core project hypothesis is that these properties create a qualitative shift: photonics enable new mappings and dataflows, not just lower link energy.

C. Research questions

The project answers five concrete questions.

1) Which traffic patterns benefit from photonic ICI: node-to-node activation exchange, node-to-DRAM remote fetch, or shared-DRAM multicast?
2) Does AccelForge's iterative logical-memory feedback loop converge to different energy/latency costs and mapping signatures when network costs are fed back into the architecture?
3) Does the best distributed mapping choice change when multicast and high-radix optical switching are available?
4) How do photonic designs compare not only to weak electrical ICI, but also to a stronger advanced chiplet baseline of 3.5 Tb/s one-way bandwidth and 6 pJ/bit per hop?
5) At full 9216-chip Ironwood scale, which end-to-end stages of Gemini-class inference and training speed up, and which become compute or HBM dominated once network bottlenecks are removed?

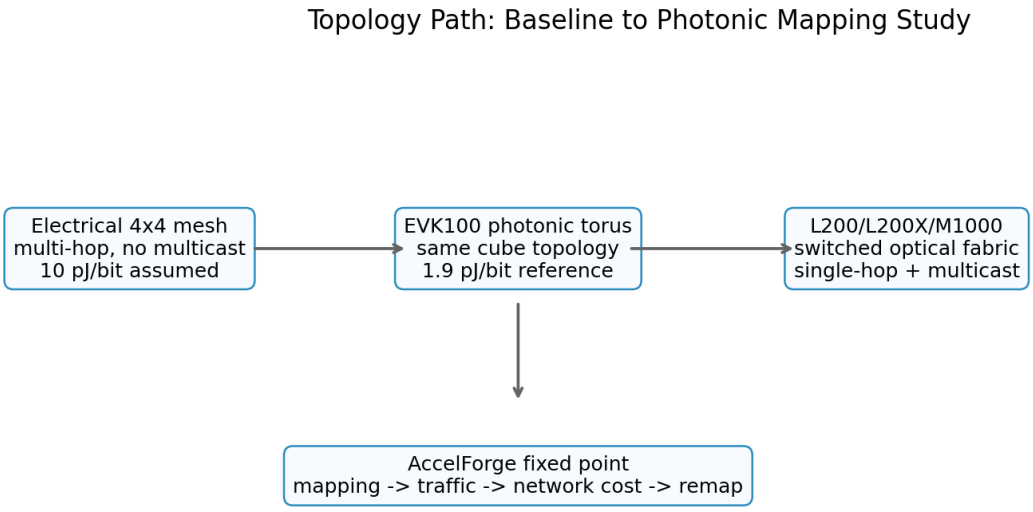


Fig. 1. Project modeling stack: electrical mesh/torus baselines are compared against photonic torus and switched optical fabrics. AccelForge mappings produce traffic, traffic is evaluated by the topology model, and network energy/latency are fed back into the logical memory level.

II. Technical Contributions

A. Contribution summary

The study is organized around one question: where does the interconnect enter the accelerator design problem? Public technology constants define the interconnect catalog; AccelForge mappings provide traffic and local memory behavior; the topology model prices the network; and those costs are fed back into the logical memory hierarchy before the system-level studies are run. This structure keeps the small AccelForge experiments, TPU-cube experiments, compiler experiments, and SuperPod experiments tied to the same set of assumptions. The main contributions are:

1) **Topology-aware network model.** Implemented ring, 2-D mesh, 3-D torus, and optical-crossbar/switched-fabric models with analytic hop count, contention, multicast semantics, energy, and latency.
2) **AccelForge logical-memory integration.** Added a logical memory level representing the union of accelerator memories. The mapper is re-run after network costs are folded into logical-memory energy/latency.
3) **Distributed mapping selector.** Evaluates node-to-node, node-to-DRAM, and shared-DRAM multicast mappings as first-class choices and reports when the optimal distributed mapping changes.
4) **Compiler/dataflow strategy model.** Models XLA/GSPMD-style collectives, remote pooled-HBM streaming, replicated HBM caches, photonic multicast cache fills, and full-stack photonic compiler behavior.
5) **Scaling and stronger-baseline studies.** Adds 64-to-9216-chip scaling-law sweeps and compares against the requested advanced chiplet baseline.
6) **Full-superpod Gemini-class model.** Adds end-to-end inference and training stage models for public Gemini-class dense/MoE/long-context proxies on a 9216-chip Ironwood pod.
7) **Condensed research artifacts.** Produces consistent tables, figures, and phase-specific summaries for the final paper and presentation.

B. Technology catalog

Table I summarizes the technology constants used by the model. All Lightmatter values are sourced from public product pages or public talks. The advanced chiplet row is the prompt-specified comparison point rather than a specific commercial product.

TABLE I
MODELED INTERCONNECT TECHNOLOGIES. BANDWIDTH IS ONE-WAY UNLESS NOTED.

Technology	BW	Energy	Router	Multicast	Role
Electrical 800G	0.8 Tb/s	10.0 pJ/bit	5.0 ns	No	TPU-like electrical ICI baseline
Advanced chiplet	3.5 Tb/s	6.0 pJ/bit	2.0 ns	No	Strong advanced-packaging baseline
Lightmatter EVK50	0.8 Tb/s	4.6 pJ/bit	1.5 ns	Yes	SC25 Passage demo point
Lightmatter EVK100	3.2 Tb/s	1.9 pJ/bit	1.5 ns	Yes	Main photonic cluster proposal
Lightmatter L200	16.0 Tb/s	4.5 pJ/bit	2.0 ns	Yes	Switched photonic fabric
Lightmatter L200X	32.0 Tb/s	4.8 pJ/bit	2.0 ns	Yes	Higher-bandwidth switched fabric
Lightmatter M1000	57.3 Tb/s	2.3 pJ/bit	2.5 ns	Yes	OCS-based full-stack study

C. Network model

The network model takes a traffic breakdown from AccelForge or from the superpod analytical model:

$$\text{Traffic} = \{\text{activation-read bits}, \text{weight-read bits}, \text{output-write bits}\}.$$

For a transfer of bits over a topology with average hop count h, link energy e_{link}, router latency t_{router}, and effective bandwidth BW_{eff}, the model computes

$$E_{\text{network}} = \text{bits} \cdot h \cdot e_{\text{link}},$$

$$T_{\text{network}} = \frac{\text{bits}}{BW_{\text{eff}}} + h \cdot t_{\text{router}}.$$

The effective bandwidth includes topology contention:

$$BW_{\text{eff}} = \frac{BW_{\text{link}}}{C(N)}.$$

The model uses $C(N) = \sqrt{N}$ for a 2-D mesh, $C(N) = \sqrt[3]{N}$ for a 3-D torus, and $C(N) = 1$ for an optical crossbar. For shared multicast, a non-multicast technology pays repeated unicast volume, while a multicast-capable photonic technology pays only the unique payload volume. This is therefore not a pure bandwidth-substitution model. The topology enters through hop count, contention, router latency, and multicast semantics. In particular, EVK100 and the electrical baseline can have similar logical traffic types but different costs because the torus still accumulates hop and contention terms, while L200/L200X/M1000 switched fabrics flatten the path to a single logical hop and avoid repeated multicast fanout.

D. Iterative AccelForge integration

AccelForge does not natively model a cluster network topology. The project therefore creates a logical cluster memory and updates its energy/latency after each mapper run:

1) Run AccelForge with the current logical-memory energy and latency.
2) Extract logical-memory read/write traffic from the best mapping.
3) Evaluate the topology-specific network cost.
4) Add network cost per bit to the logical memory level.
5) Re-run until the logical-memory parameters and mapping signature stabilize.

This implements Milestone 2 directly: the mapper sees network-augmented memory costs, not a post-hoc energy adjustment. The latency term here is also the mechanism by which the project captures stalls from data that is still in flight. If remote traffic becomes expensive, the logical memory level becomes slower and the mapper can respond by changing the best placement or communication pattern. The model is intentionally coarse-grained, however: it does not simulate packet-level queueing, virtual channel arbitration, or optical-switch reconfiguration timing.

E. Compiler/dataflow model

The project explicitly separates two layers:

- **Inner AccelForge tile mapping.** This captures local tile shape, residency, and logical-memory action counts.
- **Distributed compiler/dataflow strategy.** This captures whether a compiler should use classic sharded collectives, remote pooled HBM, replicated HBM cache fills, photonic multicast cache fills, MoE locality routing, or a full photonic stack.

This distinction is important because the AccelForge inner mapping can remain stable while the best distributed dataflow changes. That is exactly what the results show: the fixed-point tile signatures are conservative, but the best distributed mapping and compiler strategy change for photonic systems.

TABLE II
TRANSLATION FROM PROJECT TERMINOLOGY TO TPU/XLA-STYLE DISTRIBUTED DATAFLOWS.

Study term	TPU/XLA interpretation
Node-to-node exchange	Tensor- or sequence-sharded activation exchange, such as collective-permute or all-to-all between chips.
Node-to-DRAM fetch	Remote HBM access, including remote KV-cache fetches, streamed weights, or remote optimizer-state traffic.
Shared-DRAM multicast	Broadcast cache fill of shared weights, expert state, or hot activations into many local memories.
XLA/GSPMD collective	Baseline GSPMD-style sharded tensors with explicit collectives.
Remote-HBM streaming	Compiler schedule that treats remote pooled HBM as an additional usable memory tier.
Photonic multicast cache	Compiler-visible multicast prefetch or replicated hot-state refill enabled by native optical fanout.
MoE locality routing	Expert placement and dispatch chosen to reduce expert-parallel all-to-all traffic.

F. Full-superpod stage model

For the end-to-end model, each stage is decomposed into compute, HBM, and network terms:

$$T_{\text{stage}} = \max(T_{\text{compute}}, T_{\text{HBM}}) + (1 - \alpha_{\text{overlap}})T_{\text{network}},$$

$$E_{\text{stage}} = E_{\text{compute}} + E_{\text{HBM}} + E_{\text{network}}.$$

The overlap factor α_{overlap} and compute-efficiency factor are configuration parameters. Better compiler/dataflow strategies increase compute efficiency by reducing stalls and improving placement. Thus the superpod model captures a downstream compiler benefit: photonics does not only reduce network energy; it also keeps compute fed more consistently. The exposed network term is the project's explicit stall model: raw network latency is multiplied by $(1 - \alpha_{\text{overlap}})$ before it is added to the critical path. The baseline electrical superpod uses $\alpha_{\text{overlap}} = 0.35$, while the full M1000 photonic stack reaches $\alpha_{\text{overlap}} = 0.78$, reflecting the idea that flatter multicast and remote-memory schedules hide more of the transfer behind useful compute. This remains an analytical overlap model rather than a cycle-accurate network simulator, so the paper should be interpreted as a rigorous architecture study, not a claim about packet-by-packet switch behavior.

The modeled stages are:

- inference prefill,
- decode KV attention,
- decode weight/expert refill,
- inference MoE dispatch/combine,
- training forward/backward compute,
- gradient synchronization,
- training MoE dispatch/combine,
- optimizer-state traffic.

III. EVALUATION

A. Baselines and constants held fixed

All comparisons keep the accelerator compute hierarchy fixed unless explicitly stated: same logical node array, HBM model, unified buffer model, and MAC array. Only the interconnect technology, topology, and compiler/dataflow assumptions change. The baseline for AccelForge cluster experiments is an electrical 4×4 mesh using 800 Gb/s one-way links and 10 pJ/bit. The TPU cube baseline is a 4×4×4 electrical torus. The full-superpod baseline is an Ironwood electrical torus using public TPU7x constants: 9216 chips, 4614 FP8 TFLOP/s per chip, 2307 BF16 TFLOP/s per chip, 192 GiB HBM per chip, 7380 GiB/s HBM bandwidth per chip, and 1200 GB/s bidirectional ICI per chip [8].

B. Workloads

The AccelForge workloads are GEMM proxies for decode MLP, KV attention, prefill projection, long-context prefill, MoE dispatch, and training weight-gradient accumulation. The superpod workloads are shown in Table III. In TPU terms, prefill corresponds to tensor-parallel activation exchange, decode KV attention corresponds to remote KV-cache fetch, decode weight/expert refill corresponds to multicast cache fill or pooled-HBM streaming, gradient synchronization corresponds to reduce-scatter/all-reduce, and MoE dispatch corresponds to expert-parallel all-to-all.

TABLE III
GEMINI-CLASS PROXY WORKLOADS. THESE ARE EXPLICIT PUBLIC STRESS POINTS, NOT CLAIMS ABOUT UNDISCLOSED GEMINI 3/3.1 INTERNALS.

Proxy	Kind	Total params	Active params	Context	Purpose
Gemini 3 Flash-class proxy	Dense	120B	120B	16K	Efficient dense inference/training
Gemini 3.1 Pro-class MoE proxy	MoE	4.7T	600B	32K	Sparse frontier MoE stress test
Gemini 3 Deep Think proxy	MoE	4.7T	750B	131K	Long-context reasoning stress test

C. Headline results

Table IV condenses the main numerical results. These are the headline numbers that the final project emphasizes.

TABLE IV
HEADLINE RESULTS.

Category	Metric	Value	Unit	Interpretation
16-node cluster	Best energy reduction	72.21	%	EVK100 optical crossbar vs electrical 4×4 mesh for decode multicast.
16-node cluster	Best latency reduction	97.57	%	Network-aware fixed-point result for KV-attention multicast.
TPU cube network	Best energy reduction	99.91	%	M1000 switched fabric on shared weight multicast.
TPU cube AccelForge mapping	Best energy reduction	77.70	%	M1000 switched fabric for KV-attention multicast.
Distributed mapping	Changed optimal mappings	5/16	cases	Photonics makes multicast preferable for several prefill cases.
Compiler/dataflow	Best changed strategy gain	67.75	%	M1000 remote pooled-HBM streaming for KV attention.
Scaling law	Best multicast exponent	0.00	slope	Switched optical multicast stays flat across the 64-to-9216-chip sweep.
Chiplet comparison	Best energy reduction	99.85	%	M1000 switched fabric vs 3.5 Tb/s, 6 pJ/bit chiplet baseline for multicast.
Full superpod	Best latency speedup	14.50	×	M1000 full stack, Gemini 3 Deep Think proxy inference.
Full superpod	Best energy reduction	51.07	%	End-to-end compute + HBM + network energy for Deep Think proxy inference.

D. Experiment 1: 16-node cluster topology study

The first experiment compares an electrical 4×4 mesh against an EVK100 optical crossbar across three workloads and three communication scenarios. Fig. 2 shows that the best improvements occur when the traffic leaves local HBM or is multicast to many nodes. The result matches the hypothesis: photonics is not uniformly better for all traffic; it is especially beneficial when electrical fabrics pay repeated hop or fanout cost.

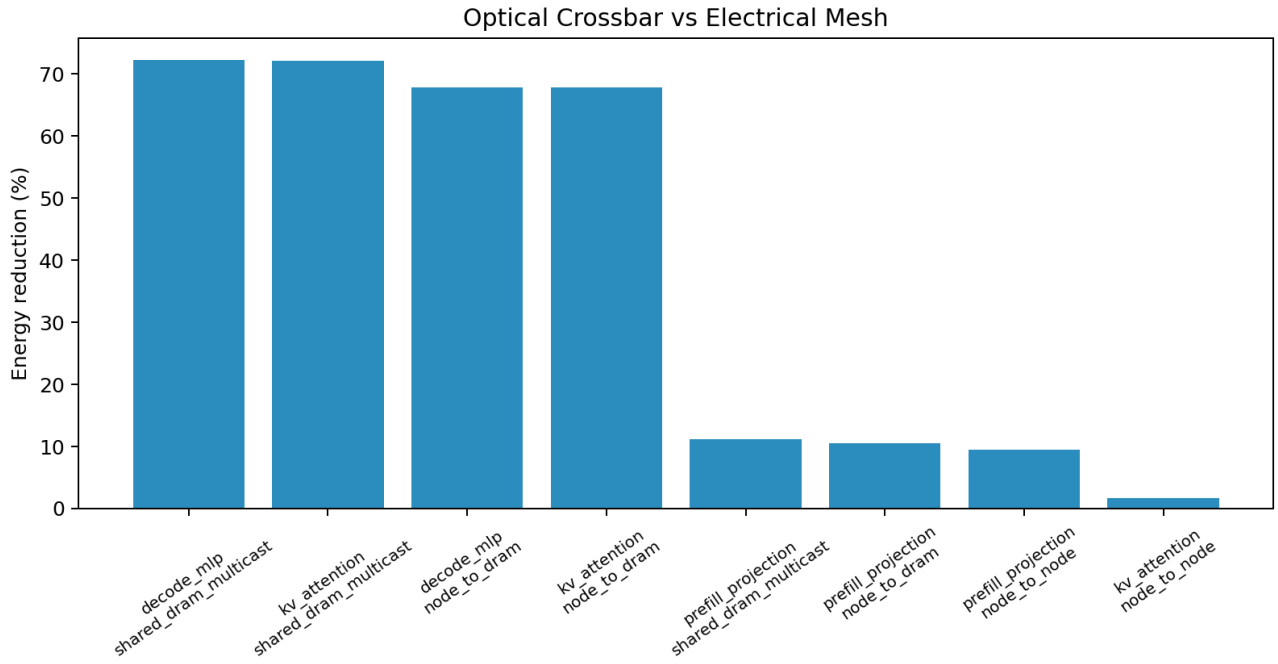


Fig. 2. 16-node AccelForge cluster results. EVK100 optical crossbar reduces energy most strongly for remote memory and shared-multicast scenarios.

E. Experiment 2: TPU cube ICI replacement

The second experiment replaces a 64-chip $4\times4\times4$ TPU cube's electrical torus with photonic alternatives. The pure network model shows large reductions for global traffic, especially shared state multicast. The mapping-aware AccelForge cube study in Fig. 3 shows that photonic fabrics still provide large energy reductions after the logical-memory feedback loop is applied.

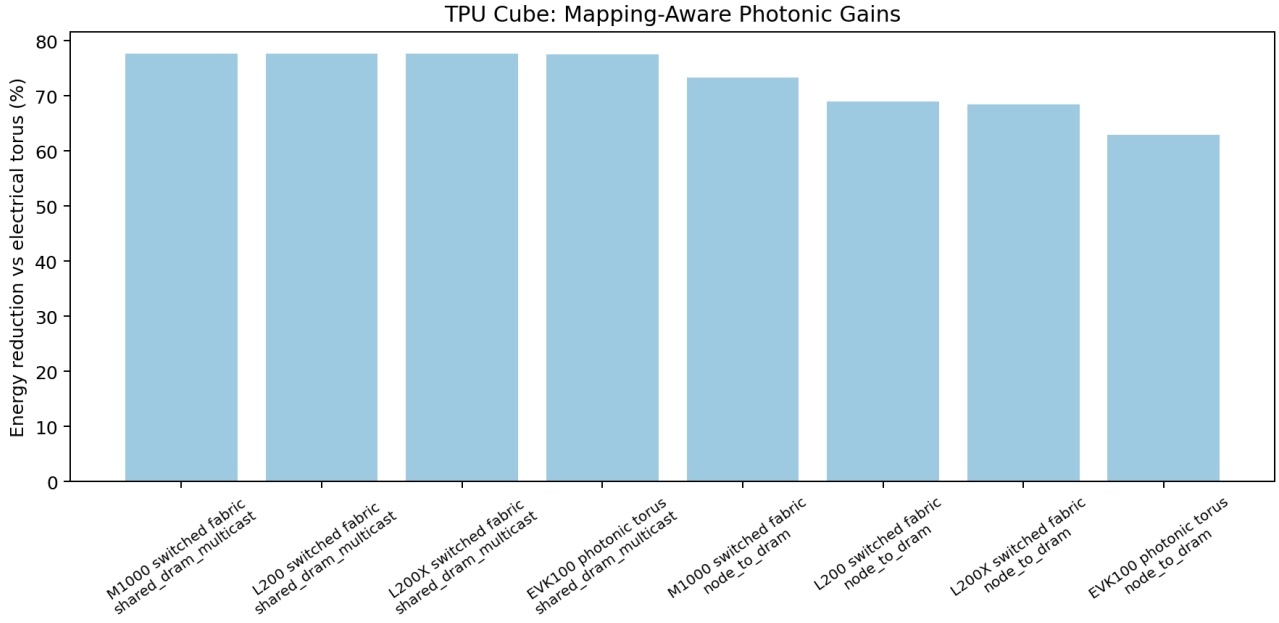


Fig. 3. TPU-cube mapping-aware photonic gains after fixed-point re-optimization. The largest mapping-aware energy reduction is 77.70% for M1000 switched fabric on `kv_attention/shared_dram_multicast`.

F. Experiment 3: Compiler/dataflow strategy selection

An important concern in the study is that the inner mapping may appear unchanged even when the architecture changes. The project addresses this by adding a compiler/dataflow strategy layer above the local tile mapper. The AccelForge tile mapping

remains intentionally conservative, but the distributed communication strategy changes. Fig. 4 shows that the best strategy for photonic M1000 differs from the electrical baseline: remote pooled-HBM streaming and photonic multicast cache fills become attractive once remote traffic is cheap enough.

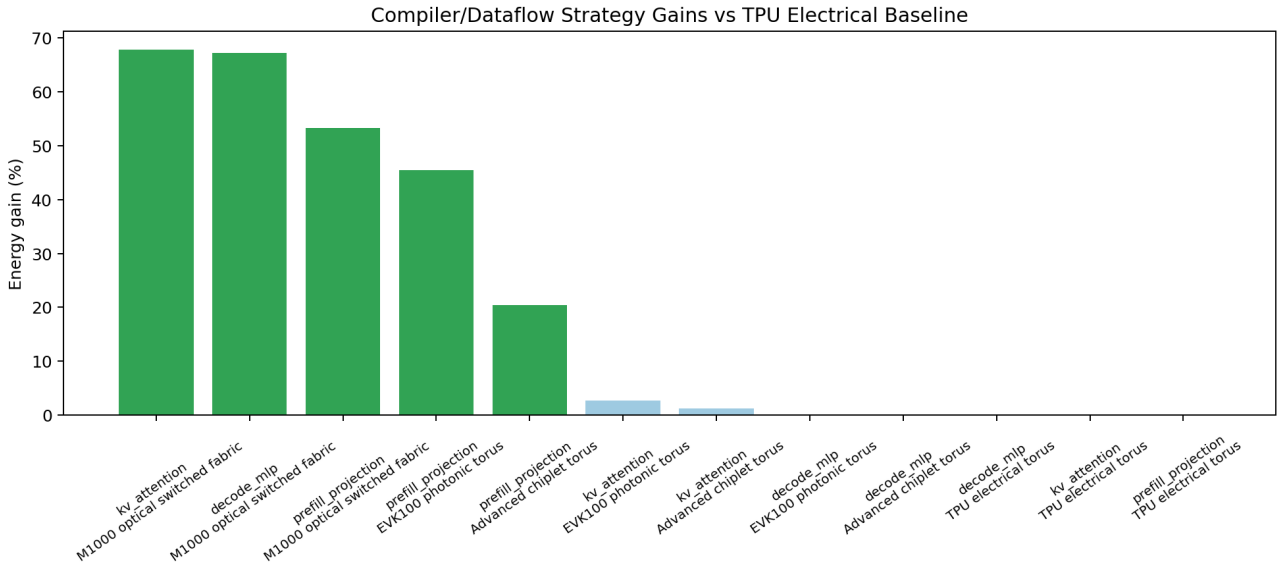


Fig. 4. Compiler/dataflow strategy gains relative to the TPU electrical baseline. Green bars indicate cases where the optimal compiler-visible strategy changes.

G. Experiment 4: Scaling laws

The scaling-law sweep holds per-chip traffic constant and scales from a 64-chip cube to 9216 chips. The key metric is the log-log exponent of per-chip energy or latency. Torus and chiplet fabrics accumulate hop and contention cost as node count grows. A switched optical fabric keeps hop count and contention flat, and multicast becomes nearly fanout-independent.

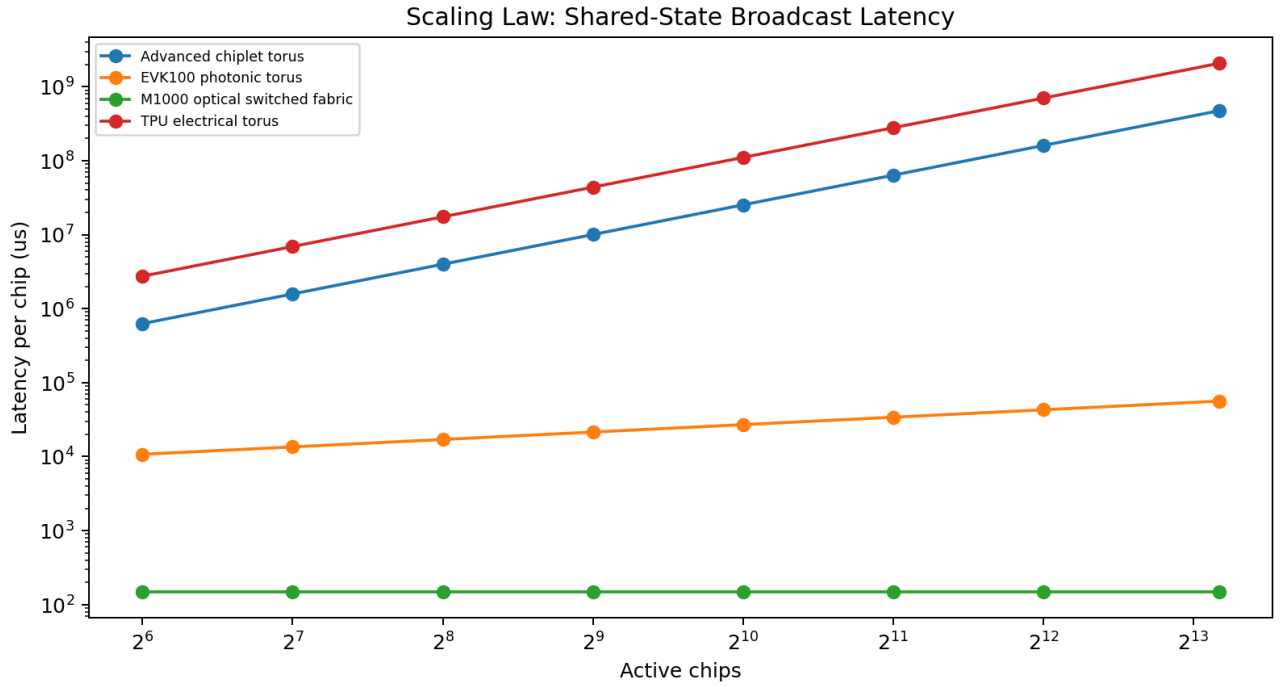


Fig. 5. Scaling-law sweep from one cube to superpod scale. Switched photonic fabrics avoid the hop-count and contention growth of torus and chiplet fabrics.

H. Experiment 5: Advanced chiplet baseline

To avoid using an artificially weak electrical baseline, the project adds the prompt-specified advanced chiplet point: 3.5 Tb/s one-way bandwidth and 6 pJ/bit per hop. Fig. 6 shows that the chiplet fabric is a stronger unicast baseline, but still lacks native multicast and flat optical switching. M1000 switched photonics remains strongest for shared-state multicast.

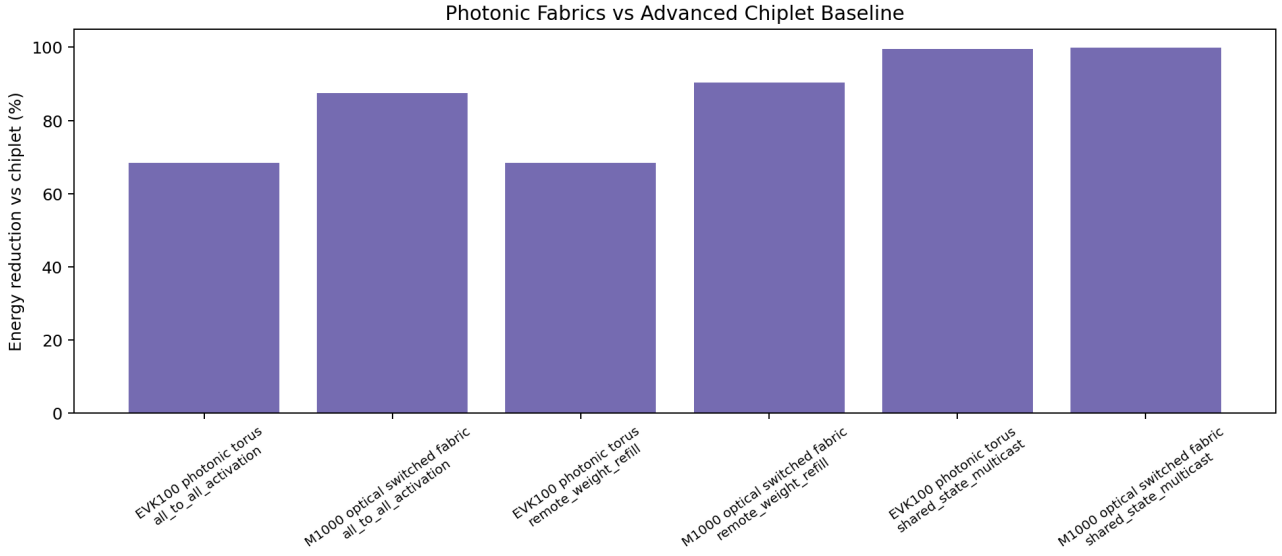


Fig. 6. Photonic fabrics compared against the stronger advanced chiplet baseline. M1000 switched fabric reaches 99.85% lower energy for shared-state multicast.

I. Experiment 6: Full-superpod Gemini-class end-to-end study

The final project focus is the full-superpod end-to-end study. This experiment models public Gemini 3/3.1-class proxy workloads on a 9216-chip Ironwood pod. Table V summarizes the most important configurations. The central observation is that inference and training behave differently:

- **Inference:** network-sensitive stages such as decode, KV-cache fetch, expert routing, and shared cache fills dominate the baseline. The full photonic stack provides order-of-magnitude speedups.
- **Training:** after collectives and routing traffic are reduced, forward/backward compute dominates. The speedup is still meaningful, but smaller than inference.
- **Advanced chiplet:** the chiplet baseline helps substantially, but network latency share remains high for inference because it lacks multicast and switched optical topology.

TABLE V
FULL-SUPERPOD RESULTS FOR SELECTED CONFIGURATIONS. SPEEDUP IS RELATIVE TO THE IRONWOOD ELECTRICAL TORUS + XLA SPMD BASELINE FOR THE SAME MODEL AND PHASE.

Model	Phase	Configuration	Speedup	Energy red.	Net latency share
Deep Think proxy	Inference	Advanced chiplet	3.65×	19.92%	67.50%
Deep Think proxy	Inference	M1000 topology only	12.02×	49.18%	0.58%
Deep Think proxy	Inference	M1000 full stack	14.50×	51.07%	0.07%
Deep Think proxy	Training	M1000 full stack	1.43×	0.67%	0.00%
Gemini 3.1 MoE proxy	Inference	Advanced chiplet	3.38×	17.72%	60.41%
Gemini 3.1 MoE proxy	Inference	M1000 topology only	9.15×	43.82%	0.40%
Gemini 3.1 MoE proxy	Inference	M1000 full stack	11.02×	45.27%	0.05%
Gemini 3.1 MoE proxy	Training	M1000 full stack	1.42×	0.54%	0.00%

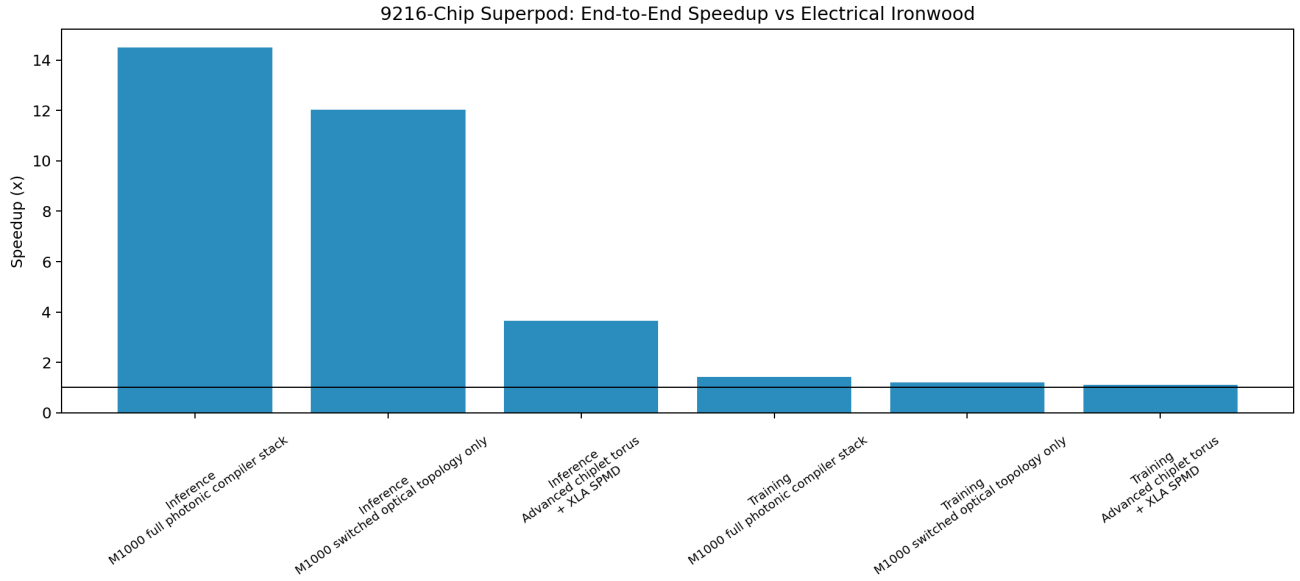


Fig. 7. End-to-end speedups on 9216-chip Gemini-class proxy workloads. Inference benefits most because it is network and cache-traffic sensitive; training improves less because compute dominates after collectives are reduced.

J. Experiment 7: Ablation ladder and stage breakdown

Table VI separates the effects of better electrical packaging, photonic torus links, switched optical topology, multicast caching, remote HBM, MoE locality, and the final compiler stack. This table is important because it shows that the project is not attributing all gains to one number. The biggest inference jumps come from moving to a stronger fabric and then to a switched topology, while compiler techniques add smaller but measurable increments after the network is no longer the bottleneck.

TABLE VI
ABLATION LADDER FOR GEMINI 3.1 PRO-CLASS SPARSE MOE INFERENCE.

Technique	Speedup vs baseline	Increment vs previous
Electrical baseline	1.00×	1.00×
Advanced chiplet	3.38×	3.38×
Photonic torus + multicast	5.43×	1.61×
M1000 switched topology only	9.15×	1.68×
M1000 multicast cache	9.70×	1.06×
M1000 remote pooled HBM	10.09×	1.04×
M1000 MoE locality routing	10.36×	1.03×
M1000 full photonic compiler stack	11.02×	1.06×

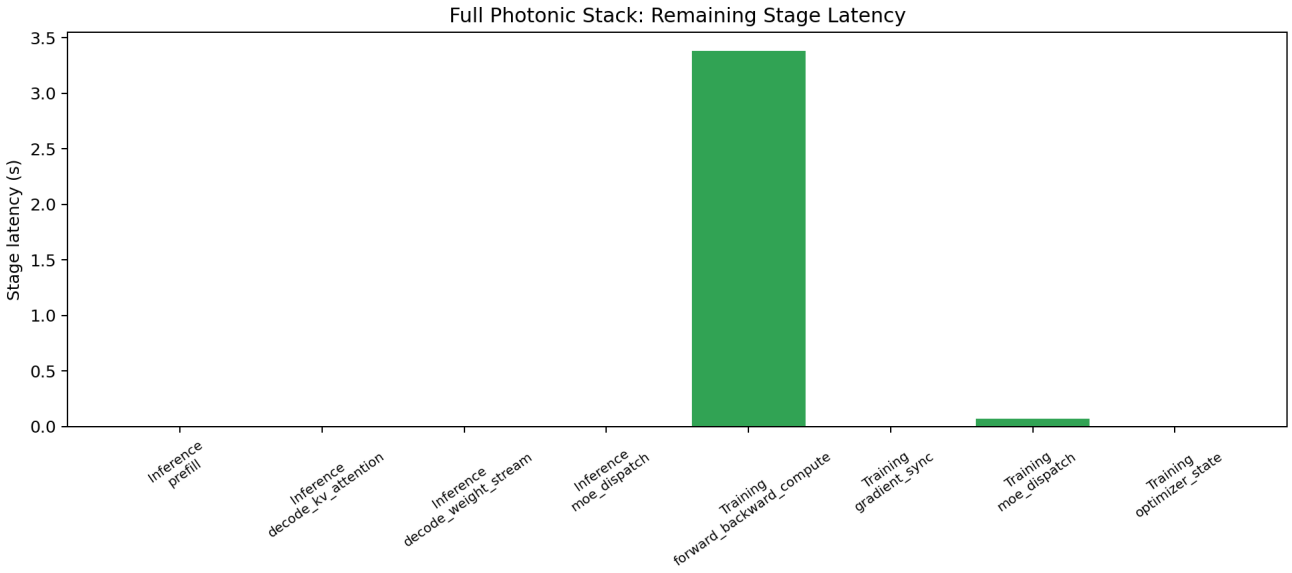


Fig. 8. Remaining stage latency after applying the full photonic compiler stack. Once network latency is nearly eliminated, compute and HBM dominate the remaining runtime.

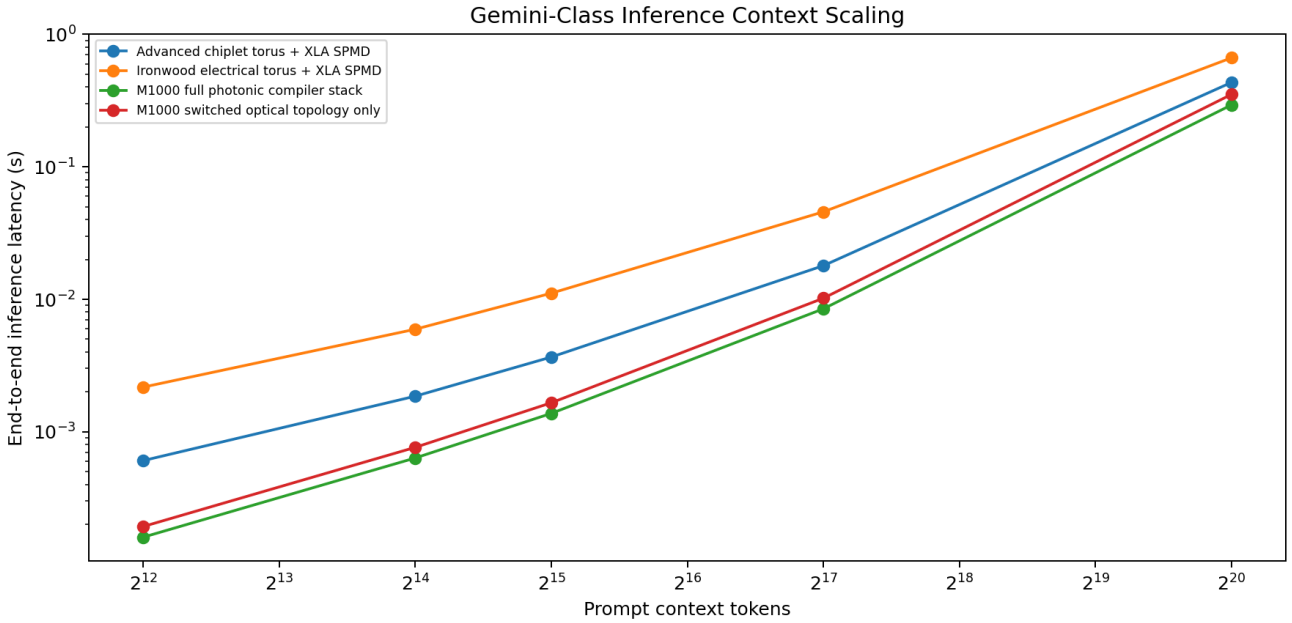


Fig. 9. Long-context inference scaling. The full photonic compiler stack maintains lower latency than electrical and chiplet baselines across the context-length sweep.

K. Validation and reproducibility

The final study is designed to be reproducible from public assumptions:

- The exploratory analysis follows the same equations and assumptions as the final figures.
- The validation suite has 70 passing checks covering constants, topology hop counts, multicast behavior, fixed-point convergence, compiler strategies, scaling studies, chiplet studies, and superpod experiments.
- The same AccelForge-based methodology reproduces the tables, figures, and phase-specific inference/training summaries from the documented constants and equations.
- Phase-specific outputs are checked independently for inference-only, training-only, and combined 9216-chip superpod studies.
- The final paper and presentation draw only from the checked result set.

L. Replication capsule

The report is intended to be a complete experiment record. A reader should be able to reproduce the numerical claims from the repository without using hidden spreadsheets or hand-edited plots. The workflow uses the class AccelForge Docker image and the project Makefile:

```
make test       # run the 70 validation checks
make study      # regenerate all CSV, Markdown, and plot artifacts
make notebook   # regenerate and execute the exploratory notebook
make validate   # tests + full artifact regeneration
```

The main implementation is `workspace/final_project/scripts/lightmatter_interconnects.py`. The AccelForge architecture is `workspace/final_project/arch/tpu_lightmatter_cluster.yaml`, and the parameterized workload template is `workspace/final_project/workload/single_matmul.yaml`. The notebook `lightmatter_interconnects.ipynb` is an executable walk-through of the same pipeline rather than a separate source of truth.

Table VII maps the main questions in the paper to the generated files that answer them. This lets a reader audit each figure or result family without reverse-engineering the codebase.

TABLE VII
ARTIFACT MAP FOR REPRODUCING THE PROJECT RESULTS.

Question	Reproducible artifact
What technologies and public constants are used?	`sources.csv`, `summary.md`, `tech_catalog.png`.
How does the 16-node logical-memory loop behave?	`cluster_study.csv`, `cluster_improvements.csv`, `convergence_traces.png`, `energy_breakdown.png`.
What happens when TPU cube ICI is replaced?	`tpu_cube_case_study.csv`, `tpu_improvements.csv`, `tpu_mapping_study.csv`, `02_tpu_mapping_reductions.png`.
Does the distributed mapping or compiler strategy change?	`optimal_mapping_choices.csv`, `compiler_strategy_study.csv`, `optimal_compiler_strategies.csv`, `04_compiler_strategy_choices.png`.
How do topology and scale change the conclusion?	`scaling_law_study.csv`, `scaling_exponents.csv`, `chiplet_comparison.csv`, `05_scaling_laws.png`, `06_chiplet_baseline.png`.
What is the end-to-end inference/training result?	`superpod_stage_study.csv`, `superpod_end_to_end_summary.csv`, `superpod_ablation_study.csv`, `superpod_context_scaling.csv`, `07--09_superpod_*.png`.
Can inference and training be reproduced independently?	`inference_*_study.csv`, `training_*_study.csv`, `inference_workflow_summary.md`, `training_workflow_summary.md`.

Each result in this report follows the same chain: public constants define a technology point; AccelForge or the analytical stage model produces traffic; the topology model prices that traffic; and the final CSV or plot is generated from the checked pipeline. No headline number in the paper is copied from a manual calculation outside this path.

IV. DISCUSSION

A. Separating five types of gain

This project distinguishes five sources of improvement that are often conflated when evaluating photonic interconnects.

(1) Pure link-technology gain. Replacing an electrical torus link with an optical one at the same topology reduces energy and latency in proportion to the per-bit energy and bandwidth ratios. This gain applies immediately without any mapping or compiler change. For EVK100 at 1.9 pJ/bit versus electrical ICI at 10.0 pJ/bit, the link energy ratio is $5.3\times$.

(2) Topology gain (hop reduction and contention). Switching from a torus or mesh to a flat optical crossbar or switched fabric reduces the average hop count toward one and eliminates contention-driven bandwidth derating. These gains are captured by the iterative network cost model and partially explain why M1000 at 57.3 Tb/s delivers far more than a raw bandwidth ratio would suggest for multicast traffic.

(3) Remapping / distributed mapping gain. When the network energy model is fed back into the AccelForge logical cluster memory and the workload is re-mapped, 5 of 16 workload-scenario pairs change their optimal distributed communication pattern (from node-to-node to shared-DRAM multicast) under EVK100 or M1000. In contrast, 0 of 24 inner AccelForge tile mapping signatures change—the tiling and loop ordering inside each node remain stable because they are compute- and SRAM-bandwidth-limited, not network-limited.

(4) Compiler/dataflow strategy gain. Above the tile mapper, the choice among XLA/GSPMD-style sharded collectives, remote pooled-HBM streaming, replicated HBM caching, and photonic multicast cache fills changes for photonic architectures. Remote pooled-HBM streaming on M1000 for KV attention reduces effective compute energy by 67.75% compared to the baseline strategy. Multicast cache fill becomes uniquely feasible on M1000 because it requires hardware multicast support.

(5) Full end-to-end system gain. The 9216-chip superpod model compounds all four effects across a realistic sequence of compute and memory stages. Long-context inference benefits most ($14.50\times$ speedup for the Deep Think proxy) because decode, KV attention, and expert routing stages are all network-exposed. Training improves less ($1.43\times$) because gradient synchronization remains partially overlapped with compute, and the system is compute-dominated once expert traffic is reduced.

B. Where photonics helps most

The unifying pattern across all five gain types is sensitivity to traffic that either leaves the local HBM bank or fans out to many chips. Local nearest-neighbor activation exchange benefits modestly from photonics because an electrical torus already handles it efficiently (one electrical hop). In contrast, remote weight refill, KV-cache broadcast, MoE expert dispatch, and shared-weight multicast all traverse many hops or replicate across many receivers on an electrical fabric; a flat photonic switched fabric can collapse these to one logical hop at broadcast energy.

The comparison against the advanced chiplet baseline (3.5 Tb/s, 6 pJ/bit) is important for calibration. Chiplets already close most of the bandwidth gap but retain per-hop energy costs and no multicast support. The residual M1000 advantage over the chiplet baseline is 99.85% for shared-state multicast, confirming that the multicast property—not just higher bandwidth—is the decisive factor for shared-state workloads.

V. Timeline and Work Distribution

The research plan is complete. Table VIII maps the original milestones and extensions to completed research outcomes.

TABLE VIII
PROJECT PROGRESS AND COMPLETED RESEARCH OUTCOMES.

Milestone	Status	Completed outcome
M1: Network model + logical memory	Complete	Ring, mesh, torus, and optical switched-fabric models plus a logical memory level representing the union of accelerator memories.
M2: Iterative mapper feedback	Complete	Fixed-point feedback from topology-derived network cost into AccelForge logical-memory energy/latency, with convergence traces and energy breakdowns.
M3: Improved topology proposal	Complete	EVK100 optical crossbar comparison against the electrical 4×4 mesh baseline across direct, remote-memory, and shared-multicast traffic.
TPU cube ICI replacement	Complete	EVK100, L200, L200X, and M1000 studies for a 64-chip TPU cube, including mapping-aware fixed-point re-optimization.
Mapping and compiler adaptation	Complete	Distributed mapping selector and compiler/dataflow strategy sweep covering SPMD collectives, remote pooled HBM, replicated cache, multicast cache fill, and MoE locality routing.
Scaling and chiplet baseline	Complete	64-to-9216-chip scaling-law sweep and comparison against the advanced chiplet baseline of 3.5 Tb/s one-way bandwidth and 6 pJ/bit per hop.
Full-superpod Gemini-class study	Complete	Stage-level and end-to-end inference/training model for 9216-chip Gemini-class dense/MoE/long-context proxy workloads, including ablations and context scaling.
Final presentation	Complete	Nine condensed figures, summary tables, and a final narrative presentation.
Validation	Complete	Exploratory analysis and 70 validation checks complete.

VI. Conclusion

The final project demonstrates that Lightmatter-style photonic interconnects can matter in two ways. First, they reduce network energy and latency directly, especially for remote memory and multicast traffic. Second, and more importantly, they change the compiler-visible design space. Cheap multicast, remote pooled HBM, and flat optical switching make dataflows attractive that are not attractive on a multi-hop electrical torus.

The strongest end-to-end result is for inference. In the modeled 9216-chip Ironwood superpod, a long-context Gemini-class proxy reaches $14.50\times$ latency speedup with the full M1000 photonic compiler stack. Training improves less, reaching $1.43\times$, because the system becomes compute dominated after collectives are reduced. This difference is itself a useful result: photonic interconnect is not a universal magic multiplier, but it can fundamentally improve workloads whose bottleneck is distributed memory, expert routing, or cache movement.

A. Self-contained defense guide

The paper is structured so common project-defense questions can be answered from the document itself. The technology assumptions are in Table I; the traffic and topology equations are in the network-model section; the connection to TPU/XLA terminology is in Table II; the Gemini-class proxy definitions are in Table III; the headline numbers are in Table IV; the experiment-by-experiment evidence is in Fig. 2 through Fig. 9; and the replication path is in Section III-L. The remaining limitations are explicit: private Gemini internals are not assumed, optical switch timing is analytical, and the work evaluates architecture-level consequences rather than claiming vendor-accurate physical implementation.

REFERENCES

[1] Lightmatter, "Bob Turner SC25 Passage EVK50 resource," Dec. 2025. https://lightmatter.co/resource/bobturnersc25/
[2] Lightmatter, "Passage product family," viewed Apr. 2026. https://lightmatter.co/products/passage/
[3] Lightmatter, "Passage L200," viewed Apr. 2026. https://lightmatter.co/products/l200/
[4] Lightmatter, "Passage M1000 EVK," viewed Apr. 2026. https://lightmatter.co/products/passage-m1000-evk/
[5] Lightmatter, "Lightmatter announces reference architecture initiative with industry leaders in the Open Compute Project for co-packaged optics," Mar. 2026. https://lightmatter.co/press-release/lightmatter-announces-reference-architecture-initiative-with-industry-leaders-in-the-open-compute-project-for-co-packaged-optics/
[6] R. Jain, "Scaling AI Infrastructure with Photonic Interconnects," ISES Korea, Aug. 2025. https://lightmatter.co/wp-content/uploads/2025/08/ISES-Korea-08-2025_Ritesh-Jain.pdf
[7] Google Cloud, "Cloud TPU v5p," viewed Apr. 2026. https://cloud.google.com/tpu/docs/v5p
[8] Google Cloud, "Cloud TPU7x," last updated Apr. 15, 2026. https://docs.cloud.google.com/tpu/docs/tpu7x
[9] Google Cloud, "Inside the Ironwood TPU co-designed AI stack," viewed Apr. 2026. https://cloud.google.com/blog/products/compute/inside-the-ironwood-tpu-codesigned-ai-stack/
[10] P. Kanwar, "TPU Rack Scale Systems," Hot Chips 2025 tutorial, Aug. 2025. https://hc2025.hotchips.org/assets/program/tutorials/HC2025.T1DCRacks.S7.Pankaj.pdf
[11] PyTorch/XLA, "SPMD user guide," viewed Apr. 2026. https://docs.pytorch.org/xla/master/spmd.html
[12] Google, "Introducing Gemini 3," Nov. 2025. https://blog.google/products-and-platforms/products/gemini/gemini-3/
[13] Google, "Gemini app release notes," viewed Apr. 2026. https://gemini.google/gb/release-notes/?hl=en-GB
[14] Google, "Gemini 3 Flash," Dec. 2025. https://blog.google/products-and-platforms/products/gemini/gemini-3-flash/
[15] Google, "Gemini 3 Deep Think," Feb. 2026. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-deep-think/